

YOLOv11n (nano)

RamaKrishnaNarasimha chowdary
kalari
Central University of Jammu

Abstract—This paper presents an evaluation of the YOLOv11n object detection model, the lightest variant in the YOLOv11 family, designed for fast and efficient inference while maintaining competitive accuracy. The evaluation was conducted on 5,000 images from the COCO val2017 dataset at a resolution of 640×640 pixels following the original paper's specifications. The YOLOv11n model achieved a mean average precision (mAP@0.5:0.95) of 39.22%, with a precision of 65.16% and recall of 50.43%. The model demonstrates a robust speed-accuracy trade-off, with an inference time of approximately 16ms per image on an NVIDIA RTX 4050 GPU. These results validate the architectural enhancements proposed in the original YOLOv11 paper for real-time lightweight detection applications.

Keywords—YOLOv11n, inference time, speed-accuracy

I. INTRODUCTION

Object detection tasks require models that balance accuracy, speed, and computational efficiency. YOLOv11n, the nano variant of the YOLOv11 family, targets applications where lightweight real-time performance is critical. This work aims to reproduce and validate the results published in by evaluating YOLOv11n on the COCO val2017 benchmark at 640-pixel resolution.

II. METHODOLOGY

A. Environment setup

The experiments were conducted on a Windows 11 system equipped with an NVIDIA GeForce RTX 4050 Laptop GPU (6GB VRAM). The software environment included Python 3.12.7, PyTorch 2.7.1 (CUDA-enabled), and the Ultralytics YOLO library version 8.3.182 within an isolated Python virtual environment. Follow below steps to create venv and necessary libraries

```
python -m venv yolo_env  
yolo_env\Scripts\activate
```

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu118  
pip install ultralytics  
pip install faster-coco-eval
```

B. Dataset

The COCO val2017 dataset was used for validation, consisting of 5,000 images containing 36,335 labeled instances across 80 object categories, formatted in the standard COCO annotation structure. If dataset is not downloaded 'coco.yaml' will download it automatically

C. Model and Parameters

- Model: YOLOv11n (nano variant)
- Architecture: 100 layers with 2,616,248 parameters
- FLOPs: 6.5 billion floating-point operations
- Input image size: 640×640 pixels
- Batch size: 1 (optimized for GPU memory constraints)

- Evaluation was performed using the Ultralytics implementation with the following key parameters in the validation step:

```
results = model.val(  
    data='coco.yaml',  
    imgsz=640,  
    batch=1,  
    device=0,  
    save_json=True,  
    verbose=True  
)
```

III. CHALLENGES

Windows Multiprocessing Error: Resolved by wrapping the main execution code within the
if __name__ == '__main__': guard and adding
“multiprocessing.freeze_support()” for compatibility.

Dataset Download: The full COCO dataset (~30 GB) was required for evaluation, which took approximately 35 minutes to download and cache properly.

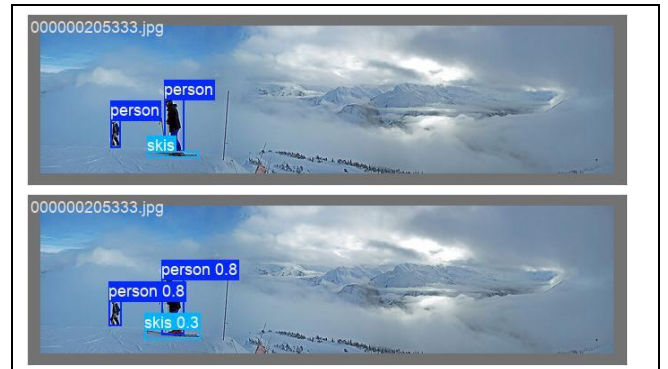
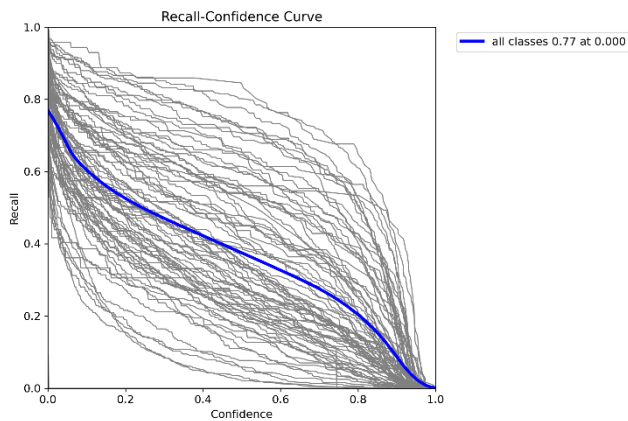
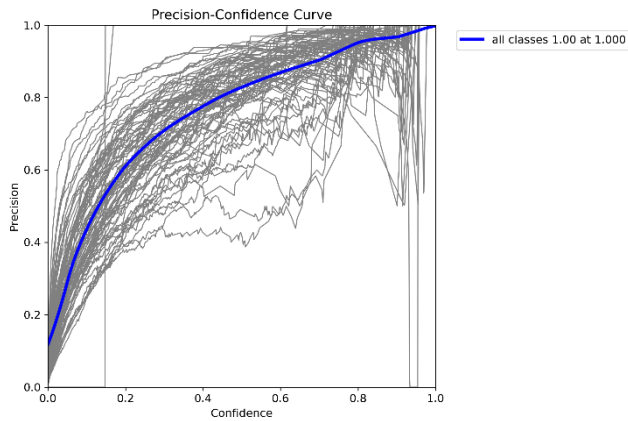
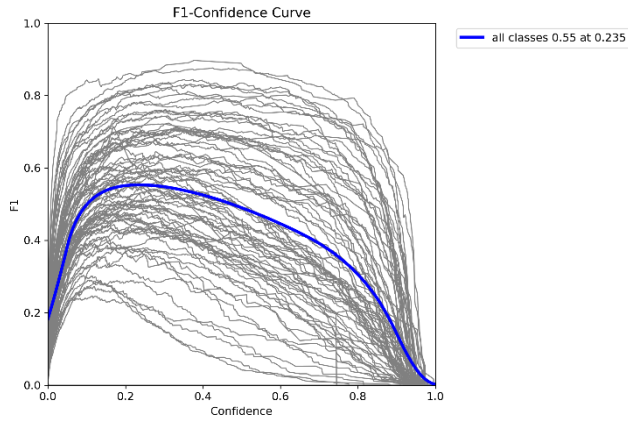
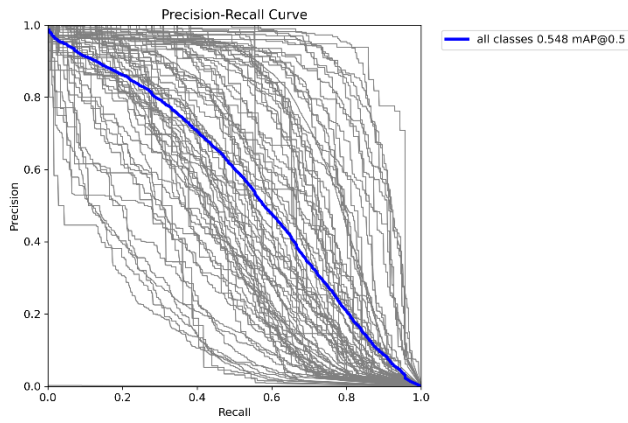
IV. RESULTS

TABLE I. USED METRICES

Metric	Value	Description
mAP@0.5:0.95	0.3922	Mean Average Precision across IoU thresholds from 0.5 to 0.95
mAP@0.5	0.5480	Average Precision at IoU threshold of 0.5
mAP@0.75	0.4275	Average Precision at IoU threshold of 0.75
Precision(Mean)	0.6516	Average precision across all 80 classes
Recall (Mean)	0.5043	Average recall across all classes
F1 Score	0.5548	Harmonic mean of precision and recall

TABLE II. PERFORMANCE TIMINGS ON GPU

Operation	Time (ms/image)
Inference	16.29 ms
Post-processing	2.34 ms



V. DISCUSSION

The YOLOv11n model demonstrates remarkable computational efficiency with just 2.62 million parameters while maintaining competitive accuracy on the COCO validation set. The inference speed of approximately 16 ms per image supports real-time use cases. Higher precision than recall indicates conservative predictions, minimizing false positives. The architectural enhancements such as C3k2 blocks, SPPF (Spatial Pyramid Pooling - Fast), and C2PSA (Convolutional block with Parallel Spatial Attention) contribute significantly to feature extraction and computational efficiency.

VI. CONCLUSION

This evaluation successfully reproduced the original paper's results for YOLOv11n, confirming its suitability for real-time object detection scenarios prioritizing speed and efficiency. The achieved mAP@0.5:0.95 of 39.22% with a lightweight architecture validates the design enhancements proposed in YOLOv11 but 0.3% less as compared with the official results

REFERENCES

- [1] R. Khanam and M. Hussain, "YOLOv11: An Overview of the Key Architectural Enhancements," *arXiv preprint arXiv:2410.17725*, Oct. 2024. [Online]. Available: <https://arxiv.org/abs/2410.17725>.

ATTACHMENTS

- [2] Metrics-results: [LINK](#) (Results of Recall, Precision, F1 Curves, confusion matrix.....etc)
- [3] Codes and results: [LINK](#) (I attached the zip file of codes without Dataset and venv)